LESSON PLAN · GENERATIVE AI (GR 6–8) INTERMEDIATE: GEN-AI CREATORS · LEVEL 2

Lesson 2: Debugging with AI

Module 1 | 60 minutes | Describe · Share · Test · Understand | Grades 7–11 · ages 12–16

Objective: Write a clear bug report for an AI — what you expected, what happened and the exact error — then test the fix and explain why the bug happened.

Success criteria

- I can write a bug report with expected, actual, error and what I tried.
- I can test an AI's fix instead of trusting it.
- I can tell an error from a logic bug.
- I can explain why a bug happened in one sentence.

Key vocabulary

- **bug**: A mistake in a program that makes it go wrong.
- **debugging**: Finding and fixing bugs.
- **error message**: Python's note about what went wrong and on which line.
- **logic bug**: The program runs but gives the wrong answer.
- **bug report**: A clear description: expected, actual, error, what you tried.
- **test**: Running the program with chosen inputs to check it works.

You will need

- Projector or screen share with Kitabash and the AI tools for this lesson
- Devices: 1 per learner (laptop, tablet or phone)
- Learner accounts only where allowed: see the course safety notes
- Printed worksheet (optional)
- Project files: l02-bug-hunt-kit.md, l02-teacher-demo-script.md

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game**	Bug detective: teams race to find the bug in three chai recipes and write a one-line bug report.
5 min	**Hook + poll**	Show print("Score: " + 5). Poll: will it print Score: 5, Score 5, or an error? Run it to reveal the TypeError.
5 min	**Watch**	Play the lesson video; pause after the failed int() fix and ask "How did Meera know it was wrong?"
7 min	**Live demo**	Teacher demo: bad prompt vs full bug report, test the AI fix live, report back, write the one-sentence reason.
7 min	**Prompt Lab (pairs)**	Pairs write the perfect bug report in the Prompt Lab and improve it until all items are ticked.

5 min	**Quick competition**	Error match and helpful-or-unhelpful sort race; first three all-correct win a shout-out.
12 min	**Breakout: fix three bugs**	In breakout rooms of two or three, learners fix the three buggy programs from the kit with AI hints and fill their bug logs. The debug loop order game for early finishers.
5 min	**Show-and-tell**	Each room shares the trickiest bug and the one-sentence reason. Did any AI fix fail? Thumbs up if yours did.
9 min	**Probe + exit ticket**	Python Lab: the broken average (solo), New situation: no error, wrong answer, then the exit ticket.

Unplugged warm-up

Bug detective (5 min): show three “recipes” on the board for making chai, each with one bug (wrong order, a missing step, “add 50 spoons of sugar”). Teams race to find the bug and write a one-line bug report: “I expected..., but...”. Link it: the better the report, the faster the fix.

Teaching the idea

Debugging is a skill, and AI can be a great debugging partner — if the learner gives it good information and tests everything. Teach the four moves: Describe (expected vs actual), Share (the exact error and line number), Test (run every suggested fix), Understand (ask why and write one sentence). AI fixes are sometimes wrong: they may change the wrong thing, or “fix” a bug in a way that hides a logic error. Learners ask for a hint first, change one thing at a time and report back honestly. Privacy rule applies to code: no passwords or real details in what they paste.

Analogy: Debugging with AI is like visiting a doctor: if you only say “I feel bad”, the doctor guesses. If you say where it hurts, since when and what you already tried, you get real help — and you still check the medicine works.

Live demo: Shared screen: Kitabash Python Lab + Kitabash AI helper. 1) Type the score bug and run it; read the TypeError aloud and underline the line number and error type. 2) First type a BAD prompt: “It doesn’t work, fix it.” Show the vague reply. 3) Type the full bug report (expected, happened, error, tried, hint first). 4) Apply the AI’s fix yourself and run it. If it suggests int(score), run it and show the same error — “Caught it! Report back.” If it suggests str() or an f-string straight away, ask: “Why does this work?” and test with score = 0 too. 5) Write the one-sentence reason on the board: “+ cannot mix text and numbers.”

Questions to ask

Ask	Listen for
What four things go in a good bug report?	What you expected, what happened, the exact error/line, what you tried.
Why ask for a hint first?	So you learn to fix it yourself and understand the bug.

The AI's fix runs with no error. Is it correct?	Not necessarily — test the output; there may be a logic bug.
What is a logic bug?	The program runs but gives the wrong answer.
Why change one thing at a time?	So you know which change fixed (or broke) it.

Differentiation

Support
Give a fill-in bug report card: "I expected ___. Instead ___. The error is ___ on line ___. I tried ___." Start learners on Bug 1 (syntax) only.

Stretch
Write your own buggy program with one hidden logic bug. Swap with a partner, who must find it using a bug report and AI hints — without being told where it is.

Common misconceptions

Misconception	How to fix it
No error message means the program is correct	Logic bugs give wrong answers with no error. Always check the output.
The AI's fix must be right	AI fixes can be wrong or fix the wrong thing. Run and test every fix.
Error messages mean I am bad at coding	Every programmer gets errors. They are clues that point to the bug.
Pasting the whole program with "fix it" is fastest	A clear report with the error and expected output gets better help faster — and you learn.

Assessment

Use the Prompt Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
An AI's fix runs with no error. What must you still check, and why?

Homework
Finish your bug log for all three programs. For each, write: the error (or wrong output), the fix, whether the AI's first fix worked, and why the bug happened.

Answer key — Kitabash activities

1. Prompt Lab: the perfect bug report (perform · ai lab)

Checklist: Say what you expected · Say what actually happened · Paste the error message · Give the line number · Say what you tried · Ask for a hint first

Strong example:

```
I am learning Python. I expected my program to print "Score: 5" but instead I get an error. The error message is: TypeError: can only concatenate str (not "int") to str, on line 3. I already tried adding a space after the colon. Please give me a hint first, not the whole answer.
```

2. Match each error to what it usually means (perform · matching)

SyntaxError → Python cannot read a line — a missing bracket or quote · NameError → A name is not defined — often a typo · TypeError → Mixing types that cannot go together, like text + number · IndentationError → The spaces at the start of a line are wrong · Logic bug → No error, but the answer is wrong

3. Helpful bug report or unhelpful? (perform · categorize)

Helpful: “I expected 77.75 but got 103.67.”, “NameError: name ‘scoer’ is not defined, line 4.”, “I already tried changing the loop to range(4).”
Unhelpful: “It’s broken, fix it.”, Pasting 300 lines of code with no question, “Just send me the right program.”

4. Python Lab: the broken average (probe · python lab)

Goals checked: Runs with no errors · Prints the correct average · Uses len() to count the marks

One solution:

```
marks = [72, 85, 90, 64]
total = 0
for m in marks:
    total = total + m
average = total / len(marks)
print("Average: " + str(average))
```

5. The debug loop (perform · ordering)

Order: Run the program and read the whole error → Write a clear bug report for the AI → Ask for a hint first → Make one change and run it again → Ask why the bug happened and write it down

6. New situation: no error, wrong answer (probe · multiple choice)

Answer: Tell the AI the exact wrong output and what he expected

Answer key — worksheet

1. **Key words:** bug = A mistake in a program that makes it go wrong. · debugging = Finding and fixing bugs. · error message = Python's note about what went wrong and on which line. · logic bug = The program runs but gives the wrong answer.
2. **Prompt Lab: the perfect bug report:** Include: Say what you expected; Say what actually happened; Paste the error message; Give the line number; Say what you tried; Ask for a hint first. Example: I am learning Python. I expected my program to print "Score: 5" but instead I get an error. The error message is: TypeError: can only concatenate str (not "int") to str, on line 3. I already tried adding a space after the colon. Please give me a hint first, not the whole answer.
3. **Match each error to what it usually means:** SyntaxError → Python cannot read a line — a missing bracket or quote · NameError → A name is not defined — often a typo · TypeError → Mixing types that cannot go together, like text + number · IndentationError → The spaces at the start of a line are wrong · Logic bug → No error, but the answer is wrong
4. **Helpful bug report or unhelpful?:** Helpful: "I expected 77.75 but got 103.67.", "NameError: name 'scoer' is not defined, line 4.", "I already tried changing the loop to range(4)." · Unhelpful: "It's broken, fix it.", Pasting 300 lines of code with no question, "Just send me the right program."
5. **Plan your program:** Goals: Runs with no errors; Prints the correct average; Uses len() to count the marks. One solution: marks = [72, 85, 90, 64] total = 0 for m in marks: total = total + m average = total / len(marks) print("Average: " + str(average))

Kitabash original curriculum. Product names (ChatGPT, Gemini, Claude, Copilot, Canva, NotebookLM, Perplexity, Teachable Machine) belong to their owners; Kitabash is not affiliated with or endorsed by them. Tools and age rules change: check each tool's current terms before class.